

```

1 %% extract_faulted_phase_features.m
2 % Step 1: Simple comparison to identify WHICH phase is faulted.
3 % Step 2: Extracts a richer feature set from ONLY the faulted phase,
4 %         windowed over time, to build a degradation/RUL model.
5 %
6 % Prerequisite: iA, iB, iC, t, Fs already in the workspace
7 % (from simOut_currents as shown earlier).
8 t = out.simout_currents.time;
9 iA = out.simout_currents.signals.values(:,1);
10 iB = out.simout_currents.signals.values(:,2);
11 iC = out.simout_currents.signals.values(:,3);
12 kt = out.simout_k.signals.values;
13
14 Fs = 1 / mean(diff(t));
15 %% STEP 1: Identify the faulted phase (simple comparison)
16 overallRMS_A = rms(iA);
17 overallRMS_B = rms(iB);
18 overallRMS_C = rms(iC);
19
20 rmsValues = [overallRMS_A, overallRMS_B, overallRMS_C];
21 phaseNames = {'A', 'B', 'C'};
22
23 avgRMS = mean(rmsValues);
24 deviation = abs(rmsValues - avgRMS);
25
26 [~, faultIdx] = max(deviation);
27 faultedPhase = phaseNames{faultIdx};
28
29 fprintf('--- Fault detection (simple comparison) ---\n');
30 fprintf('Overall RMS -> Phase A: %.3f | Phase B: %.3f | Phase C: %.3f\n', ...
31         overallRMS_A, overallRMS_B, overallRMS_C);
32 fprintf('Faulted phase identified: Phase %s\n\n', faultedPhase);
33
34 switch faultedPhase
35     case 'A', iFault = iA;
36     case 'B', iFault = iB;
37     case 'C', iFault = iC;
38 end
39
40 %% STEP 2: Windowing parameters
41 winLength_s = 0.2; % 10 cycles at 50Hz - stable RMS/FFT, decent resolution
42 overlap = 0.5; % 50% overlap between consecutive windows
43
44 winSamples = round(winLength_s * Fs);
45 stepSamples = round(winSamples * (1 - overlap));
46 nWindows = floor((length(t) - winSamples) / stepSamples) + 1;
47
48 fprintf('Window length: %.3fs (%d samples) | Step: %.3fs | Total windows: %d\n\n', ...
49         winLength_s, winSamples, stepSamples/Fs, nWindows);
50
51 if nWindows < 1
52     error(['nWindows came out as %d - your signal is too short for this window length.\n' ...
53         'Signal length: %d samples (%.3fs total) | winSamples needed: %d.\n' ...
54         'Either shorten winLength_s, or check that t/Fs loaded correctly ' ...
55         '(e.g. Fs might be wrong if t was not loaded as expected).'], ...
56         nWindows, length(t), t(end), winSamples);
57 end
58
59 %% Preallocate features (all computed on the FAULTED PHASE ONLY, unless noted)
60 Time_s = zeros(nWindows,1);
61 RMS = zeros(nWindows,1); % overall signal strength

```

```

62 Peak          = zeros(nWindows,1);    % max absolute value
63 CrestFactor   = zeros(nWindows,1);    % Peak / RMS - shape indicator
64 StdDev        = zeros(nWindows,1);    % spread/noise level
65 Kurtosis       = zeros(nWindows,1);    % "peakedness" - flags impulsive/arcing behavior
66 Skewness       = zeros(nWindows,1);    % asymmetry of the waveform distribution
67 THD            = zeros(nWindows,1);    % 3rd harmonic vs fundamental (%)
68 H3_mag         = zeros(nWindows,1);    % raw 3rd harmonic magnitude
69 Unbalance_pct  = zeros(nWindows,1);    % needs all 3 phases, kept as context feature
70 RUL_s          = zeros(nWindows,1);
71
72 totalDuration = t(end);
73
74 %% Loop through windows
75 for w = 1:nWindows
76     idxStart = (w-1)*stepSamples + 1;
77     idxEnd   = idxStart + winSamples - 1;
78
79     winFault = iFault(idxStart:idxEnd);
80     winA     = iA(idxStart:idxEnd);
81     winB     = iB(idxStart:idxEnd);
82     winC     = iC(idxStart:idxEnd);
83     winT     = t(idxStart:idxEnd);
84
85     centerTime = mean(winT);
86
87     % --- Time-domain features (faulted phase) ---
88     r      = rms(winFault);
89     pk     = max(abs(winFault));
90     cf     = pk / r;                % crest factor
91     sd     = std(winFault);
92     kurt   = kurtosis(winFault);
93     skw    = skewness(winFault);
94
95     % --- Frequency-domain features (faulted phase) ---
96     N = length(winFault);
97     Y = fft(winFault - mean(winFault)); % remove DC before FFT
98     f = (0:N-1)*(Fs/N);
99     mag = abs(Y)/N;
100
101     [~, idx_fund] = min(abs(f - 50));    % fundamental bin
102     [~, idx_h3]   = min(abs(f - 150));   % 3rd harmonic bin
103
104     fund_mag = mag(idx_fund);
105     h3_mag   = mag(idx_h3);
106     thd      = (h3_mag / fund_mag) * 100;
107
108     % --- Cross-phase context feature (needs all 3 phases) ---
109     rA = rms(winA); rB = rms(winB); rC = rms(winC);
110     avgRMSwin = mean([rA, rB, rC]);
111     unb       = 100 * max(abs([rA, rB, rC] - avgRMSwin)) / avgRMSwin;
112
113     % --- Store row ---
114     Time_s(w) = centerTime;
115     RMS(w)    = r;
116     Peak(w)   = pk;
117     CrestFactor(w) = cf;
118     StdDev(w) = sd;
119     Kurtosis(w) = kurt;
120     Skewness(w) = skw;
121     THD(w)     = thd;
122     H3_mag(w)  = h3_mag;

```

```

123     Unbalance_pct(w) = unb;
124     RUL_s(w)         = totalDuration - centerTime;
125 end
126
127 %% Assemble table
128 featureTable = table(Time_s, RMS, Peak, CrestFactor, StdDev, Kurtosis, Skewness, ...
129     THD, H3_mag, Unbalance_pct, RUL_s);
130
131 fprintf('Feature table (all phase-specific features computed on Phase %s):\n', faultedPhase);
132 previewRows = min(5, height(featureTable));
133 disp(featureTable(1:previewRows,:));
134
135 %% Save
136 writetable(featureTable, 'degradation_features.csv');
137 fprintf('\nSaved %d rows x %d columns -> degradation_features.csv\n', ...
138     size(featureTable,1), size(featureTable,2));
139
140 %% Plot each feature vs time - visual check that trends look degradation-like
141 figure;
142 featNames =
143     {'RMS', 'Peak', 'CrestFactor', 'StdDev', 'Kurtosis', 'Skewness', 'THD', 'H3_mag', 'Unbalance_pct'};
144 for i = 1:numel(featNames)
145     subplot(3,3,i);
146     plot(Time_s, featureTable.(featNames{i}));
147     title(featNames{i});
148     xlabel('Time (s)');
149 end
150 sgtitle(sprintf('Degradation features over time - Phase %s (faulted)', faultedPhase));
151 %% select_feature_fisher_ratio.m
152 % Uses Fisher's Discriminant Ratio (FDR) to pick the single best feature
153 % for tracking degradation, then fits a simple curve to that feature
154 % over time.
155 %
156 % Prerequisite: featureTable already built (from
157 % extract_faulted_phase_features.m), containing at least:
158 %   Time_s, RMS, Peak, CrestFactor, StdDev, Kurtosis, Skewness, THD, H3_mag, Unbalance_pct
159 %% Define "healthy" vs "faulty" segments (proxy: first/last 20% of run)
160 nRows = height(featureTable);
161 healthyIdx = 1 : round(0.2*nRows);           % first 20% of windows
162 faultyIdx  = round(0.8*nRows) : nRows;       % last 20% of windows
163
164 fprintf('Healthy segment: windows %d-%d (t = %.2fs to %.2fs)\n', ...
165     healthyIdx(1), healthyIdx(end), featureTable.Time_s(healthyIdx(1)),
166     featureTable.Time_s(healthyIdx(end)));
167 fprintf('Faulty segment:  windows %d-%d (t = %.2fs to %.2fs)\n\n', ...
168     faultyIdx(1), faultyIdx(end), featureTable.Time_s(faultyIdx(1)),
169     featureTable.Time_s(faultyIdx(end)));
170
171 %% Candidate features (everything except Time_s and RUL_s, which aren't features)
172 featureNames =
173     {'RMS', 'Peak', 'CrestFactor', 'StdDev', 'Kurtosis', 'Skewness', 'THD', 'H3_mag', 'Unbalance_pct'};
174
175 fisherRatios = zeros(numel(featureNames),1);
176
177 for i = 1:numel(featureNames)
178     vals = featureTable.(featureNames{i});
179
180     healthyVals = vals(healthyIdx);
181     faultyVals  = vals(faultyIdx);
182
183     % Compute Fisher's Discriminant Ratio (FDR) for this feature
184     % (This part is commented out in the original code)
185     % fisherRatios(i) = ...
186
187 end

```

```

180     mu_h = mean(healthyVals);
181     mu_f = mean(faultyVals);
182     var_h = var(healthyVals);
183     var_f = var(faultyVals);
184
185     fisherRatios(i) = (mu_h - mu_f)^2 / (var_h + var_f);
186 end
187
188 %% Display ranked results
189 [sortedRatios, sortIdx] = sort(fisherRatios, 'descend');
190 fprintf('--- Fisher Ratio ranking (higher = better degradation feature) ---\n');
191 for i = 1:numel(featureNames)
192     fprintf('%-15s FDR = %.3f\n', featureNames{sortIdx(i)}, sortedRatios(i));
193 end
194
195 bestFeatureName = featureNames{sortIdx(1)};
196 fprintf('\nSelected feature: %s (highest Fisher Ratio)\n\n', bestFeatureName);
197
198 %% Fit a degradation curve to the selected feature
199 t = featureTable.Time_s;
200 y = featureTable.(bestFeatureName);
201
202 % Simple polynomial fit (degree 2) - captures gradual monotonic trends well
203 % without overfitting to noise. Adjust degree if the trend looks more
204 % linear or more sharply curved once you see the plot.
205 p = polyfit(t, y, 2);
206 yFit = polyval(p, t);
207
208 % Goodness of fit
209 ssRes = sum((y - yFit).^2);
210 ssTot = sum((y - mean(y)).^2);
211 rSquared = 1 - ssRes/ssTot;
212
213 fprintf('Degradation curve fit: %s(t) = %.4g*t^2 + %.4g*t + %.4g\n', ...
214     bestFeatureName, p(1), p(2), p(3));
215 fprintf('R^2 = %.4f\n', rSquared);
216
217 %% Plot: raw feature vs fitted degradation curve
218 figure;
219 plot(t, y, '.', 'Color', [0.6 0.6 0.6]); hold on;
220 plot(t, yFit, 'r-', 'LineWidth', 2);
221 xlabel('Time (s)');
222 ylabel(bestFeatureName);
223 legend('Raw feature', 'Fitted degradation curve', 'Location', 'best');
224 title(sprintf('Degradation curve fit - %s (R^2 = %.3f)', bestFeatureName, rSquared));
225 grid on;
226 %% extrapolate_rul_prediction.m
227 % Takes the fitted degradation curve (from select_feature_fisher_ratio.m)
228 % and extrapolates it forward in time to predict when the selected
229 % feature crosses a failure threshold, then computes RUL at any given
230 % "current time" point.
231 %
232 % Prerequisite: p (polynomial coefficients), bestFeatureName, t, y
233 % already in the workspace from select_feature_fisher_ratio.m
234
235 %% Define failure threshold
236 % This is an ENGINEERING JUDGMENT CALL - the value of the feature at which
237 % you'd consider the machine "failed" / needing shutdown. For a toy example,
238 % a reasonable starting point is picking a value somewhat beyond your
239 % observed data range, e.g. 1.5x the max observed value. Adjust based on
240 % actual literature/standards for your feature if you have them.

```

```

241 failureThreshold = 1.5 * max(y);
242 fprintf('Failure threshold set at: %s = %.3f (1.5x max observed value)\n', ...
243     bestFeatureName, failureThreshold);
244 fprintf('You can override this manually if you have a known engineering limit.\n\n');
245
246 % Solve the fitted polynomial for the time it crosses the threshold
247 % p is [a, b, c] for  $a*t^2 + b*t + c = y \rightarrow a*t^2 + b*t + (c - \text{threshold}) = 0$ 
248 pShifted = p;
249 pShifted(end) = pShifted(end) - failureThreshold;
250
251 rootsFound = roots(pShifted);
252 realRoots = rootsFound(imag(rootsFound) == 0);           % keep only real roots
253 futureRoots = realRoots(realRoots > max(t));           % must be AFTER observed data ends
254
255 if isempty(futureRoots)
256     error(['No valid future crossing time found - the curve may not reach ' ...
257         'the threshold, or the threshold is set too high/low. Try adjusting ' ...
258         'failureThreshold.']);
259 end
260
261 t_failure_predicted = min(futureRoots); % earliest future crossing
262 fprintf('Predicted failure time: t = %.2f s\n\n', t_failure_predicted);
263
264 % Compute RUL at any "current time" point (example: at the last observed sample)
265 t_now = t(end); % example: "now" = end of your currently observed data
266 RUL_now = t_failure_predicted - t_now;
267
268 fprintf('At current time t_now = %.2f s:\n', t_now);
269 fprintf('Predicted RUL = %.2f s\n\n', RUL_now);
270
271 % Build an RUL curve across the whole observed timeline (for plotting)
272 RUL_curve = t_failure_predicted - t;
273
274 % Plot: extrapolated feature curve + threshold + failure point
275 tExtrap = linspace(min(t), t_failure_predicted*1.1, 500);
276 yExtrap = polyval(p, tExtrap);
277
278 figure;
279
280 subplot(2,1,1);
281 plot(t, y, '.', 'Color', [0.6 0.6 0.6]); hold on;
282 plot(tExtrap, yExtrap, 'r-', 'LineWidth', 2);
283 yline(failureThreshold, 'k--', 'Failure threshold');
284 xline(t_failure_predicted, 'm--', sprintf('Predicted failure: t=%.1fs', t_failure_predicted));
285 xlabel('Time (s)');
286 ylabel(bestFeatureName);
287 legend('Observed data', 'Fitted + extrapolated curve', 'Location', 'best');
288 title(sprintf('%s degradation curve, extrapolated to failure', bestFeatureName));
289 grid on;
290
291 subplot(2,1,2);
292 plot(t, RUL_curve, 'b-', 'LineWidth', 2); hold on;
293 yline(0, 'k--');
294 xlabel('Time (s)');
295 ylabel('Predicted RUL (s)');
296 title('RUL estimate over observed timeline');
297 grid on;
298

```